

AccsMarkets -- Complete Deep-Dive Audit Report

Date: 2026-09-16 **Scope:** Full codebase (read-only analysis, no code changes made) -- 1657-line Prisma schema (75 models), ~223 API route files, 112 pages across public/dashboard/admin, 133 shared components, all `src/lib` business logic, all deployment/config/scripts. **Method:** Parallel deep-read audit across 6 top-level workstreams (which further split into 17 total sub-audits) -- database, public frontend, dashboard frontend, admin panel + authorization, core business logic, integrations/background jobs, and tests/config/cross-cutting concerns. Every finding below is backed by a specific file path and, wherever practical, a line number, verified by actually reading the code -- not inferred from filenames or page titles.

Read this first: `README.md` and `MILESTONE_2_PLAN.md` are **stale and materially wrong** about what's built. They describe dispute resolution, TOTP 2FA, WebAuthn, achievement badges, reviews, KYC, and subscriptions as "deferred to Milestone 2." **All of these are fully implemented in the current code.** Conversely, the README claims hCaptcha protects signup -- it does not; verification is hard-stubbed to always pass. Do not use the README as a source of truth for feature status; this document supersedes it.

A. Executive Summary

What this project is: A production, feature-rich escrow marketplace for buying/selling social media accounts (Next.js 14 App Router, TypeScript, Prisma/MySQL, NextAuth, Socket.IO, custom `server.js`). It is materially larger and more complete than its own README suggests -- 75 database models, 223 API routes, a full admin console (42 pages), a full user dashboard (31 pages), a full public marketing/SEO site (39 pages), a developer API portal, an AI blog-automation pipeline, and a multi-rail payment system (crypto via NOWPayments, manual USDT, bank wire, Stripe card scaffolding).

Overall maturity: mid-to-late-stage production build with a few load-bearing gaps. The core money-moving path (list -> offer -> fund escrow -> submit credentials -> verify -> transfer -> complete/dispute -> payout) is real, transactionally sound in the places that matter most, and thoroughly built out on both client and server. The weak points are concentrated in **authorization depth** (admin RBAC and 2FA), a **handful of specific broken flows** (bank-wire withdrawal, hCaptcha, one escrow state-machine bypass), and **operational hygiene** (zero automated tests, stale deployment configs, silent-failure logging gaps).

Strongest areas: - The escrow state machine and money-moving transactions (fund, complete, cancel, dispute-resolve) are correctly wrapped in `$transaction` with in-transaction re-checks against race conditions. - Dispute resolution is a complete, phased system (evidence -> review -> mediation/ruling -> appeal) with a full audit trail -- despite docs claiming it doesn't exist yet. - The listing creation wizard, escrow detail page, and messaging system are the most thoroughly built features in the app -- real-time, validated, error-handled, no shortcuts found. - Credential encryption (AES-256-GCM) is implemented correctly with no hardcoded keys and proper buyer/status-gated access. - 92% of API routes that accept a JSON body validate it with Zod.

Weakest areas: - **Admin authorization has a critical hole:** admin password login silently bypasses TOTP 2FA entirely, and the granular staff-permission system (`StaffRole`) is enforced on well under 20% of the routes it should gate -- a staff account scoped to "blog only" can, today, ban users, move money, and change platform settings. - **hCaptcha is a no-op** despite real credentials being configured -- signup has zero actual bot protection, and the frontend `<Captcha>` component isn't even rendered on the register page. - **Bank-wire withdrawal is completely non-functional** -- a payload/schema mismatch means every submission gets HTTP 400. This is a real, live money-feature gap, not a cosmetic bug. - **Zero automated tests** exist anywhere in the codebase for a live financial marketplace. - Deployment configuration is split across three contradictory pictures (Windows/PM2 -- the real one; Docker Compose; Vercel) and two of the three are confirmed stale/dead, including a fully-configured AI blog pipeline with no working trigger.

Biggest missing functionality: Subscription cancellation/downgrade (no path exists at all once a user upgrades), enforced granular admin permissions, working bank-wire withdrawal, real bot protection on signup, and an automated test suite.

Biggest production risks (ranked): 1. Admin accounts effectively cannot be protected by 2FA against password compromise (auth.ts:113). 2. Any admin-role staff account has unrestricted access regardless of the permissions the

owner assigned them. 3. Wallet debit paths (escrow purchase, withdrawal approval) use read-then-write balance arithmetic instead of atomic decrements -- a real (if unconfirmed-in-production) double-spend race under concurrent load. 4. Signup has no working bot/abuse protection. 5. Bank-wire withdrawals silently fail for every user who tries them.

B. Project Architecture

- **Framework:** Next.js 14 (App Router), TypeScript, custom Node server (`server.js`) -- **not** Vercel serverless, because Socket.IO requires a long-lived process (this is explicit and correct in the README, but contradicted by a stale `vercel.json`).
- **Database:** MySQL 8.4 via Prisma ORM, 75 models, single "baseline" migration (no incremental history retained).
- **Auth:** NextAuth with a credentials provider (email/password + TOTP), Google OAuth, and a separate WebAuthn/passkey credentials provider scoped to the admin subdomain.
- **Real-time:** Socket.IO, mounted on the same HTTP server as Next.js (`server.js` + `socket-server.js`), used for messaging, escrow status pushes, and admin live views. Explicitly designed to **no-op safely** if disconnected -- REST is the source of truth, sockets are a convenience layer only.
- **Payments:** NOWPayments (crypto, HMAC-SHA512-verified, idempotent), manual USDT (admin-confirmed), bank wire (admin-confirmed, receipt/invoice generation), Stripe (fully coded but the `stripe` npm package isn't installed and no keys are configured -- completely inert today).
- **Storage:** Google Drive (primary) with Cloudinary fallback for uploads; Google Drive also used for hourly/daily DB backup uploads.
- **Background processing:** BullMQ + Redis is fully optional -- the app is designed to gracefully fall back to direct/synchronous processing when Redis is absent (which it currently is in production). A separate `worker.js` process exists for when Redis *is* configured, but has no process-supervisor entry for the real (non-Docker) deployment.
- **Admin subdomain routing:** `admin.accsmarkets.org` and `maintenance.accsmarkets.org` are handled by host-based logic in `src/middleware.ts`, rewriting to `/admin/*` and `/maintenance` respectively.
- **Real deployment topology (verified this session and by the audit):** PM2 process manager + Windows Task Scheduler watchdog scripts on a bare Windows Server VM, fronted by a Cloudflare Tunnel.
`docker-compose.yml` / `Dockerfile` and `vercel.json` are both **stale, unused artifacts** of alternate deployment plans that were never kept in sync with reality (confirmed: `docker-compose.yml` references the wrong encryption-key env var name; `vercel.json` 's cron targets a route that only accepts `POST`, while Vercel Cron calls via `GET`, and relies on a `CRON_SECRET` that's never set).
- **Third-party content platforms:** Opiny (external CMS for `/resources/*`, Svix-signature-verified webhook) and a custom "AutoSEO" blog-sync webhook -- two distinct content pipelines, separate from the in-house AI blog-generation system (Groq/Grok/OpenAI/Gemini fallback chain).

C. Complete Existing Feature List

Legend: [OK] Fully Implemented - [PARTIAL] Partial - [MISSING] Missing - [BROKEN] Implemented but Broken - [MOCK] Prototype/Mock - [NEEDS WORK] Needs Improvement

Module	Feature	Status	Frontend	Backend	Database	Prod Ready	Notes
Auth	Email/password login + logout	[OK]	<code>login/page.ts</code>	<code>src/lib/auth.User</code> , <code>LoginAttempt</code>			5-attempt/15-min lockout via DB table, real
Auth	Google OAuth	[OK]	login/register pages	NextAuth google provider	Account	Y	Standard, auto-verifies

Module	Feature	Status	Frontend	Backend	Database	Prod Ready	Notes
Auth	Email verification (mandatory)	[OK]	verify-email/	api/auth/verify	VerificationToken		Real token flow
Auth	TOTP 2FA (setup/verify/disable)	[OK]	settings/security	api/auth/2fa	TwoFactorAuth	Y	Real RFC 4226/6238 math. Enforced at login for regular users only -- bypassed entirely for ADMIN role (see Security Section!)
Auth	WebAuthn/password (admin only)	[OK]	admin/login/page	src/lib/auth.ts, api/admin/webauthn/*	WebAuthnCredential		README wrongly calls this "Milestone 2"
Auth	Password reset	[OK]	forgot-password	api/auth/forgot	Verification	Y	no Anti-enumeration (same response either way); [BROKEN] client silently shows success even on fetch failure/rate-limit (no res.ok check)
Auth	hCaptcha bot protection	[BROKEN] Broken	Captcha.tsx (widget, real keys)	(src/lib/hcaptcha.ts		N	verifyCaptcha() hard return true. Register page never even renders <Captcha>. Zero real bot protection despite configured keys
Onboarding	Buyer/seller intent wizard	[OK]	onboarding/page	api/onboarding	User	Y	Deliberately fails open (navigates on save error)
Listings	Create (3-step wizard)	[OK]	ListingWizard	api/listings	Listing	Y	Most complete flow in the app; real ownership verification, auto-moderation, draft autosave
Listings	Ownership verification (YouTube/Telegram API)	[OK]	OwnershipVer	api/listings --		Y	Manual platforms -> admin review flag

Module	Feature	Status	Frontend	Backend	Database	Prod Ready	Notes
Listings	Auto-moderation scoring	[OK]	--	src/lib/moder	Listing.status		Blocks >=40 score, re-runs on every edit
Listings	Edit (5-tab form, forces re-review)	[OK]	EditListingF	api/listings	Listing	Y	
Listings	Delete	[OK]	listings page	api/listings	Listing.DELETE	Y	Only DRAFT/PENDING/REJECTED deletable
Listings	Admin approve/reject/suspend	[OK]	admin/listin	api/admin/li	Listing, Admin	Y	Transactional, audited, IndexNow ping on approve
Listings	Browse/search/filter	[OK]	listings/page	direct Prisma (server component)	Listing.tsx	Y	Private listings correctly excluded from public browse
Listings	Plan-based listing limits	[OK]	--	api/listings	Subscription	Y	Enforced against live DB value, not the display-only plan-features.ts
Listings	Private listings + invites	[OK]	PrivateListin	api/listings	PrivateListingInvite		Full GET/POST/DELETE
Listings	Auctions (bidding)	[OK]	listing detail	api/listings	AuctionBid	Y	No DB-level guard against tied top bids (app-logic only)
Listings	Saved searches + alerts	[OK]	saved-search	api/saved-se	SavedSearch	Y	
Listings	Watchlist	[OK]	watchlist/pa	direct Prisma	Watchlist	Y	
Listings	"Wanted" buy-requests	[OK]	wanted/page.t	api/wanted-li	WantedListing		
Offers	Send/accept/decline	[OK]	OfferList.ts	api/offers,	Offer	Y	Accept auto-declines other pending offers

Module	Feature	Status	Frontend	Backend	Database	Prod Ready	Notes
Offers	72h auto-expiry	[PARTIAL]	--	lazy-check on read + api/internal/sweep	Offer	Partial	Lazy-checked on every read/action; the proactive sweep endpoint exists and is production-ready code but is not confirmed to be running on any active schedule on the live box (see SectionJ)
Offers	Chat-embedded offers	[OK]	ChatOfferCardMessageThread	api/messages	Message	* Y	
Escrow	Full state machine	[PARTIAL]	EscrowStepper	src/lib/escrow 10 routes	EscrowStateMachine	Mostly Y +	One route bypasses the state machine entirely (see SectionE)
Escrow	Fund/checkout	[OK]	CheckoutForm	api/escrows	Escrow, Transaction	Y	Transactional, in-tx re-check
Escrow	Credential submit + AES-256-GCM handoff	[OK]	EscrowAction	api/escrows/submit	Escrow	Y	Correct crypto, buyer/status-gated decrypt
Escrow	Manager-add flow (YouTube-style transfer)	[OK]	EscrowAction	submit-manager	Escrow, Escrow	Y	ail nfirm-handover
Escrow	Trustless admin handover	[OK]	admin escrow page	api/admin/escrowid/execute-trustless-handover			less-handover
Escrow	Complete	[OK]	EscrowAction	api/escrows/	Escrow, Transaction	Y	Transactional double-release guard
Escrow	Cancel (FUNDED/AWAITING_MANAGER_ADD)	[OK]	EscrowAction	api/escrows/	Escrow, Transaction	Y	Full refund incl. fee, correct
Escrow	Split-payment milestones	[BROKEN] Broken transition	MilestoneChecker	api/escrows/	EscrowMilestone	N	Release route sets VERIFYING -> COMPLETED bypassing awaited transition confirmed state-machine violation

Module	Feature	Status	Frontend	Backend	Database	Prod Ready	Notes
Escrow	Dispute (open/evidence/mediation/phases/rulings)	[OK]	DisputeEvidence, Dispute pages	api/escrows/	Dispute, DisputeEvidence	DisputeEvidence	DisputeTimeline phased system; README wrongly calls it unbuilt. Gap: deadlines not auto-swept
Escrow	Reviews	[OK]	ReviewForm.tsx	api/reviews	Review	Y	@unique([escrowId, ...]) duplicates
Escrow	Trust score (+5 on completion)	[NEEDS WORK]	--	3+ completion routes	User.trustScore	But one-directional	No penalty path for disputes lost/cancellations
Escrow	Achievement badges	[OK]	AchievementBadges	src/lib/badges	UserAchievements	Y	Real aggregation, seller-only by design
Messaging	Real-time DMs	[OK]	MessageThreadList, ConversationList.tsx	api/messages/	Message	Y	REST-first, socket is best-effort convenience only -- no message loss on disconnect
Messaging	Escrow chat	[OK]	embedded in escrow detail	api/escrows/	Message	es Y	
Messaging	File attachments	[OK]	MessageThreadList	api/upload/message		Y	
Messaging	Admin canned responses	[OK]	admin support tools	api/admin/canned	CannedResponse	Y	
Notifications	In-app + push (web + Expo mobile)	[OK]	bell dropdown, notifications.tsx	src/lib/notifications	Notification, PushSubscription	Y	Push-first, survives offline. Gaps: offer-cancel, withdrawal-request, deposit-creation don't notify
Wallet	Balance, transaction history	[OK]	WalletClient	direct Prisma	Transaction	Y	
Wallet	Crypto deposit (NOWPayments)	[OK]	DepositWidget	api/wallet/deposit	CryptoWallet, NOWPayments	Y	4096-bit MAC-SHA512 verified, idempotent
Wallet	Manual USDT deposit	[OK]	DepositWidget	api/wallet/deposit-admin-confirm	CryptoWallet	Y	Idempotent, rate-limited
Wallet	Bank-wire deposit	[OK]	DepositWidget, tracking page	api/wallet/deposit-bank-transfer	BankTransferOrder	Y	Mark-sent, invoice, admin verify all real

Module	Feature	Status	Frontend	Backend	Database	Prod Ready	Notes
Wallet	Card deposit (Stripe)	[BROKEN] Inert	Elements form (hidden if no key)	api/wallet/d	FiatPayment	N	1 Fully coded but stripe package not installed, no keys -- always 503
Wallet	Crypto withdrawal	[OK]	withdraw/page	api/wallet/wi	Transaction	Y	Rate-limited 3/hr; debit deferred to admin approval (transactional)
Wallet	Bank-wire withdrawal	[BROKEN] Completely broken	withdraw/page tab	api/wallet/w route)	Transaction	N	Payload the form sends never matches the schema the route validates against -- every submission is HTTP 400
Wallet	Admin deposit/withdrawal approval	[OK]	admin deposits/withdrawals pages	api/admin/de	Transaction, AdminAuditLog	Y	Transactional, idempotent, audited
Subscriptions	View plans, upgrade (balance or crypto)	[OK]	settings/sub	api/payments	Subscription	Y	
Subscriptions	Cancel / downgrade	[MISSING] Missing	none	none	--	N	No path exists anywhere once upgraded (see SectionD)
Organizations	Create, invite/remove members, KYB	[OK]	organization	api/organiza	Organization	Y	Backend member role-change route exists with no UI to trigger it
Referrals	Code, milestones, rewards	[OK]	referrals/pa	api/referrals	ReferralCode	Y	Referral
Settings	Profile, currency, sign-out	[OK]	settings/pag	api/user/me	User	Y	
Settings	Notification preferences	[OK]	settings/noti	api/settings	User Notifications	Y	Rare example of a proper loading+error UI
Settings	Privacy (GDPR export/erasure)	[OK]	settings/pri	api/user/exp	DataExportRe	Y	ErasureRequest

Module	Feature	Status	Frontend	Backend	Database	Prod Ready	Notes
Settings	Sessions (list/revoke)	[OK]	settings/sessions	api/auth/sessions	ActiveSession	Y	Confirm-dialog gated revoke-all
Settings	Identity verification (email + KYC upload)	[PARTIAL]	settings/verify	api/verify	KycSubmission	Partial	1 Email+ID/selfie fully built; the "phone" step is dead code -- real phone-verify endpoints exist but are never called from this page; the KYC level is actually advanced by email alone
Developer Portal	API keys, webhooks, third-party apps	[OK]	developer/packages	api/developer	ApiKey, WebhookEndpoint	Y	Apply real CRUD
Admin -- Users	Ban/unban, balance adjust, trust override, badges	[OK]	admin/users	api/admin/users	User, Admin	Y	Audited, but not permission-gated (see SectionI)
Admin -- Listings	Moderation queue	[OK]	admin/listings	api/admin/listings	Listing	Y	
Admin -- Escrows	Monitor, manual verify/transfer	[OK]	admin/escrow	api/admin/escrow	Escrow	Y	
Admin -- Disputes	Full mediation/ruling console	[OK]	admin/disputes	api/admin/disputes	Dispute	Y	
Admin -- Finance	Deposits, withdrawals, bank accounts, pricing	[OK]	admin/deposits	api/admin/*	Transaction	Y	1k Functional, but not permission-gated ; bank-account changes unaudited
Admin -- Staff/RBAC	Staff accounts + permission assignment	[NEEDS WORK]	admin/staff	api/admin/staff	StaffRole, UserRole	Y (enforcement)	1k It is real; the permissions it assigns are enforced by <20% of routes
Admin -- Settings	Platform config, maintenance mode, feature flags	[OK]	admin/settings	api/admin/settings	PlatformSettings	Y (UI)	1e Settings/feature-flag changes unaudited ; feature-flag (see SectionF)

Module	Feature	Status	Frontend	Backend	Database	Prod Ready	Notes
Admin -- Content	Blog CMS + AI generation, announcements, email templates	[OK]	admin/blog, a	api/admin/b	BlogPost, Announcement, EmailTemplate		Real-time pipeline; no working scheduled trigger on the live deployment (see SectionJ)
Admin -- Audit	Audit log viewer	[NEEDS WORK]	admin/audit-	direct Prisma	AdminAuditLog	Y (UI), Partial (data)	Viewer works; underlying log only captures ~50% of mutating admin actions
Public site	Marketing, browse, blog, legal, SEO landing pages	[OK]	39 pages under (public)	mostly direct Prisma (server components)	various	Y	Uniformly well-built; only one is an intentional stub (Careers, honestly labeled "not hiring yet")
Public -- Status page	System status	[BROKEN] Misleading	status/page.	api/health	--	N/A	4 "service" rows all mirror one single aggregate DB check -- can show false "Operational" for Socket.IO/escrow if only DB is actually up
Public -- API docs	Developer docs page	[BROKEN] Mostly fictional	docs/page.tsx-		--	N/A	Documents 7 /v1/* endpoints; 5 of 7 have no backing route
Backups	Hourly DB dump -> Google Drive	[OK] (fixed this session)	--	scripts/back	--	.t Y	Was not scheduled at all before this session; now registered and verified working
Background -- Sweep	Offer expiry, escrow flags, referral rewards, saved-search alerts	[PARTIAL]	--	api/internal/sweep	scripts/sweep, scripts/saved-search	Y running	Code is production-ready; no scheduled task for it currently exists on the live box (verified via schtasks this session)

Module	Feature	Status	Frontend	Backend	Database	Prod Ready	Notes
Background -- AI blog cron	Scheduled blog generation	[MISSING] No working trigger	--	api/cron/blog	BlogTopicQueue	N	Fully coded/configured; only trigger (vercel.json) is dead on this deployment target
Testing	Automated tests (unit/integration/e2e)	[MISSING] Missing entirely	--	--	--	N	Zero test files, zero test framework in package.json, confirmed by direct search

D. Missing Features

Priority	Module	Missing Feature	Current State	Required Work	Business Impact
Critical	Admin Auth	Enforce 2FA on admin password login	2FA exists but is hardcoded to skip for <code>role === "ADMIN"</code>	1-line-cause, real-effort fix: remove the <code>user.role !== "ADMIN"</code> short-circuit in <code>src/lib/auth.ts</code> , verify admin login page's existing TOTP UI still matches	Highest-privilege accounts are one leaked password away from full compromise
Critical	Admin RBAC	Enforce <code>StaffRole</code> permissions across all sensitive routes	Permission catalog + UI exist; enforcement is opt-in per-route and missing on most financial/user-management routes	Systematically pass the correct <code>Permission</code> argument to <code>requireAdminRole()</code> in Section I; consider making it required (no-arg call becomes a compile error)	Any admin-role staff account has unrestricted access regardless of assigned scope
Critical	Wallet	Fix bank-wire withdrawal	Form submits fields the shared endpoint's schema doesn't accept; always 400s	Either branch <code>api/wire</code> to <code>api/withdraw</code> and split into a dedicated bank-withdrawal endpoint/schema mirroring the deposit side's pattern	Users literally cannot withdraw via bank wire; a shipped, discoverable feature silently fails every time
Critical	Auth	Real hCaptcha verification	<code>verifyCaptcha()</code> to always pass; frontend widget never rendered on register	Remove the stub, call the real hCaptcha siteverify API; render <code><Captcha></code> in <code>register/page.tsx</code> and use <code>captchaToken</code> state	Signup has zero bot/abuse protection despite the operator believing otherwise

Priority	Module	Missing Feature	Current State	Required Work	Business Impact
High	Subscriptions	Cancel / downgrade	Only upgrade flows exist; the only "Cancel" button aborts an in-progress crypto payment, not the subscription	New POST <code>/api/payments/cancel</code> reverts <code>SubscriptionPlan</code> renewal boundary; UI in <code>settings/subscriptions</code>	Users on a paid plan have no self-service way to stop playing/downgrade -- support burden + potential billing disputes
High	Operations	Register the sweep task on the live server	<code>api/internal/sweeps/scripts/sweep.mjs</code> production-ready; no scheduled task currently exists (confirmed via <code>schtasks</code> this session)	Register a Windows Scheduled Task (mirroring the pattern used for the hourly backup fixed this session) hitting the sweep endpoint every 10-15 min	Offer 72h expiry, escrow transfer-deadline flags, referral rewards, and saved-search alerts are not being proactively processed -- only lazily on next read, which for a low-traffic listing means they may never flip at all
High	Testing	Automated test suite (at minimum: escrow state machine, fee calc, wallet debit/credit, auth)	None exist	Introduce Vitest/Jest; start with pure-function unit tests (<code>fees.ts</code> , <code>escrow-state-machine.ts</code> , <code>credentials-crypto.ts</code>) then integration tests against a test DB for the money-moving API routes	A financial marketplace with zero regression protection on escrow/wallet logic is a constant risk for every future change
High	Verification	Real SMS phone verification (or remove the misleading "phone" step)	Backend routes exist (<code>api/verification</code>) but are never called; the "Verify Identity" step actually only does email	Either wire the existing phone-verify routes into <code>wallet.verifySMS</code> provider (Twilio et al. -- currently none integrated at all), or delete the dead code/UI copy that implies phone verification happens	KYC level naming/UX implies stronger verification than actually occurs; also dead code maintenance burden
Medium	Admin	Audit-log coverage for financial/settings/compliance actions	~50% of mutating admin routes write to <code>AdminAuditLog</code> ; bank accounts, platform settings, feature flags, KYB decisions are unaudited	Add <code>auditLog()</code> calls to the ~13 identified unaudited route files (SectionI has the list)	Compliance/forensic gap -- sensitive changes (e.g., who changed the platform's crypto wallet addresses) have no trail
Medium	Observability	Error tracking / APM (e.g. Sentry)	None integrated; only a homegrown console-log + <code>api/client-errors</code>	Add Sentry (or equivalent) for both server and client; wire existing <code>logger.ts</code> calls and uncaught-exception paths into it	Production incidents are currently only debuggable via raw log files, no alerting

Priority	Module	Missing Feature	Current State	Required Work	Business Impact
Medium	Deployment	Reconcile deployment configs	<code>docker-compose.yml</code> and <code>vercel.json</code> are both stale/wrong relative to the real PM2/Windows deployment; several scripts reference the pre-rename <code>pathsmarkets.org</code>	Either delete the unused Docker/Vercel configs (if genuinely abandoned) or fix them; fix the remaining stale-path scripts identified in Section F	Confusion for any future operator/developer; the stale-path scripts are silently non-functional today
Medium	Dispute resolution	Auto-advance dispute phases on evidence/mediation deadline	Phase machine exists but nothing sweeps deadline expiry	Extend <code>api/dispute/sweep</code> to <code>Dispute.evidence</code> fields and auto-transition per <code>dispute-phases.ts</code>	Disputes could sit indefinitely in a phase if a party never acts and no admin manually intervenes
Low	Dashboard	Fix the always-zero "Saved" listings count	Code references a <code>SavedListing</code> Prisma model that doesn't exist in the schema; silently caught, always renders 0	Either add the model (if "saved" should be distinct from <code>Watchlist</code>) or point the count at <code>Watchlist</code> instead	Cosmetic metric, but a broken number displayed to every buyer
Low	Public site	Fix or remove the <code>/sitemap.xml</code> link and the fictional <code>/v1/*</code> API-docs endpoints	<code>sitemap/page.tsx</code> to a nonexistent XML sitemap; <code>docs/page.tsx</code> documents 5 endpoints that don't exist	Generate a real <code>sitemap.xml</code> (or remove the link); trim the docs page to the 2 real <code>/v1/*</code> endpoints or build the missing ones	Broken link (minor SEO/UX); public developer docs currently overpromise a v1 API that's 70% fictional
Low	Status page	Real per-service health checks	All 4 displayed "services" mirror one aggregate DB check	Add independent checks for Socket.IO and escrow-processing health, or reduce the page to what's actually monitored	Could show false "all operational" during a partial outage

E. Partial / Broken Features

Priority	Feature	Problem	Evidence	Recommended Fix
Critical	Admin TOTP 2FA	Silently bypassed for all admin accounts on password login	<code>src/lib/auth.ts:113</code> <code>tfa hardcoded null</code> when <code>role === "ADMIN"</code>	Remove the role-based short-circuit; enforce identically to regular users

Priority	Feature	Problem	Evidence	Recommended Fix	
Critical	Bank-wire withdrawal	Every submission returns HTTP 400	src/app/(dashboard)/src/app/api/wallet/withdrawal route.ts:19-24 against withdrawSchema (src/lib/validation/wallet.ts:15-19) which requires network + address	Branch server route on method, or split endpoints	thdraw/pa
Critical	hCaptcha verification	Always passes regardless of token	src/lib/hcaptcha.ts:1-4 true unconditionally) + register/page.tsx <Captcha>	Implement on-site verify call; render the widget and wire captchaToken	
Critical	Escrow state machine (milestone path)	Bypasses assertTransition entirely, performs an illegal transition VERIFIED -> COMPLETED	src/app/api/escrows/	Route the final-milestone completion through assertTransition /the same helper every other completion path uses	stoneIO
High	Admin RBAC enforcement	requireAdmin() skips the permission check unless a permission argument is explicitly passed; most sensitive routes don't pass one	src/lib/admin.ts:14 of call sites in src/app/api/admin/* agent findings (users/[id], withdrawals/[id], deposits/[id], bank-accounts, settings, listings/[id], escrows/[id], disputes/[id] resolve)	Make the permission argument required at the type level, or add a lint rule; audit and fix each listed route	
High	Wallet debit paths (escrow purchase, withdrawal approval)	Read-then-write balance arithmetic instead of atomic {decrement}, inside \$transaction but without a row lock -- theoretical lost-update race under concurrency	src/app/api/escrows/	Switch to {decrement: amount} mirroring the {increment} pattern already used for credits	rc/app/ap
High	Subscription cancellation	No code path exists	SubscribePlanGrid.ts only aborts an in-progress crypto payment	Build the "Cancel" endpoint/UI (see SectionD)	
Medium	Cloudinary/upload size validation	JSON/data-URI upload path never checks file size (only the FormData path does)	src/app/api/upload/:37-42	Add the same 10MB check to the data-URI branch	30 vs
Medium	Withdrawal request "reserve" check	Plain read-then-write outside any transaction; two concurrent requests can both pass the balance check	src/app/api/wallet/withdrawal route.ts:32-56	Wrap \$transaction with an in-tx re-check, or accept the current mitigation (approval step re-checks) as sufficient and just document it	

Priority	Feature	Problem	Evidence	Recommended Fix	
Medium	PromoRedemption reuse	Redemption is consumed via DELETE rather than a status flag; deleting the row removes the constraint blocking re-redemption	src/app/api/escrows/PromoRedemption	Mark redemption as consumed (status/flag) instead of deleting the row; verify does not properly prevent replay	unique (
Medium	Forgot-password client error handling	Fetch failures/rate-limits are silently treated as success	src/app/(public)/forgot-password/res.ok check)	res.ok /try-catch pattern already used correctly on the contact form	79 referenc
Low	Dashboard "Saved" count	Always renders 0	src/app/(dashboard)/a nonexistent SavedListing model	See SectionD	
Low	Public API docs page	Documents mostly-fictional /v1/* surface	src/app/(public)/docs/v1/*	See SectionD	
Low	Status page	Misleading multi-service display backed by one check	src/components/...Status page)	See SectionD	c
Low	Verification "phone" step	Dead state/UI, orphaned backend routes	src/app/(dashboard)/settings/verification/phone/*	See SectionD: 149 (phone	

F. Hidden / Unfinished Code

Genuine stub markers (rare -- this codebase is unusually clean of literal TODO/FIXME comments): -

src/lib/hcaptcha.ts:1-4 -- explicit comment // Captcha temporarily disabled -- re-enable by removing this early return above the always-true stub. The single most important "hidden" finding in the whole audit. -
src/app/api/settings/marketing-emails/route.ts:11 -- // both handlers use raw queries instead of the typed client for now. -- a genuine acknowledged tech-debt comment.

Dead code (fully written, zero call sites): - src/lib/gemini.ts -- standalone Gemini API client, superseded by the multi-provider fallback chain in src/lib/ai.ts, never called from anywhere else. -

src/components/admin/AdminDisputeActions.tsx (89 lines), AdminProfileClient.tsx (746 lines), AdminSettingsForm.tsx (197 lines), AdminStatCard.tsx (69 lines) -- all fully built, superseded by inline reimplementations in DisputePhaseControls.tsx / AdminSettingsClient.tsx. -
src/components/dashboard/DashboardHeader.tsx (220 lines), src/components/social/ActivityFeedCard.tsx (84 lines), src/components/wallet/TransactionTable.tsx (52 lines) -- same pattern: complete, working, unused v1 implementations left behind after inline rebuilds. -

src/app/(dashboard)/dashboard/settings/verification/page.tsx:149 -- phone state declared and never used again; the real api/verification/phone/* routes it should call are themselves unused by any frontend.

Scaffolded-but-inert database models: - FeatureFlag -- full admin CRUD UI + isFeatureEnabled() helper exist, but the helper is **never called anywhere in the product**. Flags can be created/toggled but gate nothing. - NpsResponse -- write-only. POST /api/nps accepts submissions; no admin page or route anywhere reads them back out.

Backend-ahead-of-frontend (API exists, no UI trigger): - PATCH /api/organizations/[id]/members/[userId] (role change) -- fully implemented, no button/control anywhere in organization/page.tsx calls it. -
api/verification/phone/send-code + /confirm -- implemented, unused (see above).

Frontend-ahead-of-backend (UI exists, backend can't fulfill it): - Bank-wire withdrawal form (see SectionE -- the form's payload shape has no matching backend support). - <Captcha> component + captchaToken state on the register page -- present in code but never rendered/wired, so the backend's verifyCaptcha() call always receives an empty string

(moot today only because that function is itself stubbed to always return true).

Stale/incorrect documentation-as-code: - README.md / MILESTONE_2_PLAN.md claiming dispute resolution, TOTP, WebAuthn, badges, reviews, KYC, subscriptions are unbuilt -- all are real. README also claims "hCaptcha on signup" works -- it doesn't. - .env comment referencing scripts/run-sweep.ps1, which doesn't exist (actual files are sweep.bat / sweep.mjs / setup-sweep-task.bat). - docker-compose.yml env var ENCRYPTION_KEY doesn't match the real CREDENTIALS_ENCRYPTION_KEY used everywhere else in the app -- proof this file has never actually been run against the real codebase. - Several deployment scripts still reference the pre-rename accsmarkets.org folder path and would fail if run today: ecosystem.config.js, start-prod.ps1, scripts/sweep.bat, scripts/backup.bat, scripts/run-marketing-sweep.ps1, scripts/setup-backup-task.bat, scripts/setup-sweep-task.bat. (watchdog-website.ps1, watchdog-mysql.ps1, watchdog-tunnel.ps1, and run-backup.ps1 were already corrected and verified working during this session's infra work.)

G. Workflow Gap Analysis

Offer lifecycle

Current: Buyer sends offer -> seller accepts/declines/counters -> Chat/notification fires -> [X] (*proactive 72h expiry not confirmed scheduled*) -> Buyer proceeds to checkout on accept *Expected:* Buyer sends offer -> seller acts -> **sweep job actively expires stale offers every ~15 min, platform-wide** -> checkout on accept *Gap:* The sweep endpoint and script exist and are well-written; the Windows Scheduled Task that should invoke it every 15 minutes is not currently registered (verified directly this session). Today, an offer only expires when *someone* happens to load GET /api/offers or attempt a PATCH on it.

Escrow milestone completion

Current: Buyer releases final milestone -> route directly sets Escrow.status = COMPLETED -> [X] (*state machine's own rule that VERIFIED cannot go directly to COMPLETED is silently violated*) *Expected:* Buyer releases final milestone -> route calls assertTransition(VERIFIED, IN_TRANSFER) -> ... -> assertTransition(IN_TRANSFER, COMPLETED), matching every other completion path *Gap:* One specific route doesn't use the shared state-machine guard at all.

Admin staff permission scoping

Current: Owner creates a staff account, assigns e.g. "Content only" permissions in the UI -> [X] (*most routes never check the permission*) -> staff account can still ban users / move money / change settings *Expected:* Owner assigns "Content only" -> every sensitive route checks requireAdmin("MANAGE_X") -> staff account is actually restricted to content actions *Gap:* The enforcement layer exists (requireAdmin(permission)) but is opt-in per-route and used on a small minority of routes.

Signup bot protection

Current: User fills registration form -> submits -> [X] (*no captcha ever collected*) -> server "verifies" an empty token -> always passes -> account created *Expected:* User fills form -> completes a real hCaptcha challenge -> token sent -> server calls hCaptcha's siteverify API -> only proceeds on real success *Gap:* Both ends of this flow are individually broken (frontend never renders the widget; backend never actually checks it).

Bank-wire withdrawal

Current: User fills bank details -> submits -> server 400s (schema mismatch) -> user sees a generic error toast, has no way to actually withdraw via bank wire *Expected:* User fills bank details -> server validates a bank-specific schema -> creates a PENDING bank-withdrawal request -> admin reviews and approves/rejects like the existing deposit/crypto-withdrawal flows *Gap:* The whole backend acceptance path for this specific payment method is missing; only the crypto path was actually finished on the server side.

Subscription lifecycle

Current: Free -> Upgrade (balance or crypto) -> [OK] Active paid plan -> [X] (*no further lifecycle step exists*) *Expected:* Free -> Upgrade -> Active -> Cancel/Downgrade request -> reverts to Free at next renewal boundary (or immediately, per business decision) -> confirmation + notification *Gap:* The entire back half of the subscription lifecycle (cancel/downgrade) was never built.

H. Frontend / Backend / Database Gap Matrix

Frontend exists, backend missing or broken: - Bank-wire withdrawal form -> backend schema rejects it unconditionally. - Register page's Captcha component -> never rendered, so the real (if it were fixed) backend check would never receive a token anyway.

Backend exists, frontend missing: - `PATCH /api/organizations/[id]/members/[userId]` (role change) -- no UI control. - `api/verification/phone/send-code` + `/confirm` -- no UI step calls these. - `MANAGE_FINANCE` / `MANAGE_USERS` permission constants -- declared and shown in the staff-role assignment UI, but literally zero route in the codebase checks for them, so assigning a staff member "Finance" or "Users" scope in the admin UI currently has no restrictive effect at all (they already have full access as any ADMIN would).

Database exists, functionality incomplete/missing: - `FeatureFlag` -- full schema + admin CRUD, zero product-code consumption. - `NpsResponse` -- write path only, no read/reporting surface anywhere. - `SavedListing` referenced by application code but **doesn't exist in the schema at all** -- inverse of the usual gap (code assumes a table that was never created).

Navigation exists, feature incomplete: - `settings/subscription` -> "Cancel" only cancels an in-flight payment, not the subscription itself; a user landing here expecting to downgrade has no path. - `settings/verification` -> "Verify Identity" step visually implies phone verification but is actually just email confirmation renamed. - Public `/docs` page -> links out an API surface that's 70% non-existent. - Public `/sitemap` page -> links to a `/sitemap.xml` that returns 404.

I. Security Findings

Critical

1. **Admin TOTP 2FA is silently bypassed on password login.** `src/lib/auth.ts:113` hardcodes `tfa = null` whenever `user.role === "ADMIN"`, so the entire TOTP-enforcement block is dead for admin accounts regardless of whether they've enabled 2FA. The admin login page still renders a TOTP input for a `TOTP_REQUIRED` error the server can never emit via this path. Only the separate WebAuthn provider actually enforces a second factor. **Any admin with a leaked password logs in with password alone.**
2. **hCaptcha verification is a hardcoded no-op.** `src/lib/hcaptcha.ts:1-4` -- `verifyCaptcha()` always returns `true`. Real `HCAPTCHA_SITE_KEY` / `HCAPTCHA_SECRET_KEY` are configured in `.env`, so whoever operates this site believes signup is bot-protected; it is not. Compounded by `src/app/(public)/register/page.tsx`, which imports `<Captcha>` and declares `captchaToken` state but never renders the widget or calls `setCaptchaToken` -- the token sent to the server is always an empty string.
3. **Granular admin permissions (`StaffRole`) are enforced on a small minority of sensitive routes.** `requireAdmin(permission)` only performs the permission check when a caller passes one (`src/lib/admin.ts:14`); dozens of financially/operationally sensitive routes call it with no argument, including user ban/balance-adjust (`api/admin/users/[id]`), withdrawal/deposit approval, bank-account CRUD, platform settings, and (inconsistently vs. its own sibling routes) dispute resolution. **`MANAGE_FINANCE` and `MANAGE_USERS` -- the two most sensitive permission constants in the catalog -- have zero enforcing call sites anywhere in the codebase.** A staff account the owner explicitly scoped away from Finance/Users can still perform every Finance/Users action.

High

4. **`/api/admin/**` has no middleware-level authorization at all**, by design of `src/middleware.ts` (both the admin-subdomain and main-domain branches explicitly skip `/api/*`). Currently safe only because all 81 admin route files independently self-check role -- verified route-by-route -- but there is no structural safety net; a future route added without remembering the check would be fully public with no second line of defense.
5. **Wallet debit paths use non-atomic read-then-write arithmetic** instead of Prisma's atomic `{decrement}` (which credits already correctly use via `{increment}`). Sites: `src/app/api/escrows/route.ts:159-165` (escrow purchase

debit), `src/app/api/admin/withdrawals/[id]/route.ts:55-60` (withdrawal approval debit). Both are inside `$transaction`, but a plain `SELECT` inside a MySQL transaction under Prisma's default isolation does not take a row lock -- two concurrent requests against the same user's balance could both read the same starting value and the second write clobbers the first. Not confirmed exploited; a real code-pattern risk under concurrent load.

6. **No rate limiting on any authenticated admin mutation route**, including money-moving ones (`withdrawals/[id]`, `deposits/[id]`, `bank-transfers/[orderId]`, user balance-adjust, dispute resolution, trustless handover) and -- most notably -- **admin 2FA-setup TOTP code verification has no attempt limit at all**, making it brute-forceable by anyone holding a valid (even non-2FA-enabled) admin session during the setup window.

Medium

7. **AdminAuditLog covers roughly half of mutating admin actions**. Confirmed unaudited despite sensitivity: platform bank-account create/update/delete, platform-wide settings changes (including crypto deposit addresses and fee windows), feature-flag creation/toggling, KYB approve/reject (uses `logger.info` instead), escrow-manager-email pool changes, transfer policies, canned responses, email templates, and an admin enabling/disabling their own 2FA.

8. **Cloudinary/upload data-URI path has no server-side file-size check** (`src/app/api/upload/listing/route.ts:19-30`), unlike its sibling `FormData` path which correctly enforces a 10MB cap -- a large base64 payload can bypass the limit entirely through this code path.

9. **CREDENTIALS_ENCRYPTION_KEY in the live .env is 66 hex characters**, not the 64 (32-byte) length `src/lib/credentials-crypto.ts` expects/requires. This needs empirical verification (not confirmed broken in this read-only audit) -- if the running code path actually enforces the 64-char check and the live value doesn't match, escrow credential encryption could be failing or silently using a truncated/different key than intended. **Flagged for immediate verification, not confirmed as an active incident**.

10. **Admin page components have no per-page authorization check of their own** -- `src/app/(admin)/admin/layout.tsx:46-47` renders children directly when there's no session at all (only redirects when a session exists but has the wrong role), relying entirely on middleware having already caught the unauthenticated case. No second line of defense for that specific state.

Low

11. Public API docs page describes a largely fictional `/v1/*` surface -- not itself a vulnerability, but publishing documentation for endpoints that don't exist could mislead integrators or probing tools.

12. Client-side-only error swallowing on `forgot-password` (silently reports success even on server error/rate-limit) -- low-severity information/UX issue, not an auth bypass (the server-side logic itself is correct and anti-enumeration-safe).

No evidence found of: SQL injection (Prisma parameterizes everything; the one raw-SQL usage in the blog-detail page is parameterized), XSS (content is sanitized via `src/lib/sanitize.ts` before render in the places checked), missing webhook signature verification (NOWPayments HMAC-SHA512, Stripe signature, Opinly Svix -- all verified correctly), or hardcoded secrets in application source (secrets live in `.env`, not committed code -- though `prisma/seed.ts` does hardcode and print a known admin password, see SectionF/SectionJ).

J. Reliability & Data Integrity Findings

- **Wallet debit race condition** (detailed in SectionI #5) -- theoretical double-spend/lost-update under concurrent escrow purchases or withdrawal approvals for the same user.
- **Offer/escrow sweep not confirmed scheduled** -- verified directly this session via `schtasks /query` that no "AccsMarkets Internal Sweep" (or any sweep-related) task exists on the live server, despite `scripts/sweep.mjs` + `setup-sweep-task.bat` being written specifically for this purpose. In practice this means offer 72h expiry, escrow transfer-deadline flags, referral-milestone rewards, and saved-search alerts are currently **only evaluated lazily**, on whatever page load or action happens to trigger the same-request check -- a low-traffic listing's offer could sit expired-in-spirit but not-yet-flipped indefinitely.

- **Dispute model has zero database indexes** (`prisma/schema.prisma:524-547`) beyond the implicit unique on `escrowId` , while the primary admin moderation query (`api/admin/disputes-list/route.ts`) filters by `status` and sorts by `createdAt` -- this will become a full-table scan as dispute volume grows (contrast: `Escrow` and `Report` both correctly have `@@index([status])`).
- **PromoRedemption consumed via row deletion rather than a status flag** -- since the anti-replay protection is a unique constraint on the row itself, deleting it on use removes the very thing that would block a second redemption of the same code by the same user, unless `/api/promo/route.ts` independently guards against replay (not verified in this pass).
- **Duplicate-pending-offer check is not race-safe** (plain `findFirst + create` , no transaction or unique constraint) -- two concurrent submissions from the same buyer on the same listing could create two `PENDING` offers. No money moves at this stage, so impact is limited to data cleanliness/UX confusion.
- **Money stored as Float in three config-level (not core-ledger) locations:**
`PlatformSettings.bankTransferShortfallToleranceUsd` , `DepositMethodFee.feeRate/minFee/maxFee` , `SubscriptionPlan.escrowFeeRate` -- all consumed directly in real dollar comparisons/calculations. The actual ledger fields (`walletBalance` , `Escrow.amount` , `Transaction.amount/balanceBefore/balanceAfter` , etc.) are correctly `Decimal` , so this is a lower-blast-radius issue than a `Float` ledger would be, but still a real rounding-precision risk in the specific flows that use these fields (bank-transfer auto-verification tolerance, deposit fee bounds, escrow fee-rate math).
- **internal/sweep has no top-level try/catch** despite running 6 sequential DB-mutating steps per invocation -- an unexpected error mid-sweep silently skips all later steps in that run with no record of why (Next.js returns a generic 500, but there's no application-level log of which step failed).
- **Very low structured-logging adoption** (`src/lib/logger.ts` used in only 7 files/13 call sites) versus 74 files with `try/catch` -- most non-critical failures are caught and silently discarded (`.catch(() => null)` / `.catch(() => [ ])`), which is defensible for fire-and-forget side effects (notifications, activity events) but means production issues in those paths are invisible without adding logging.
- **No automated tests** means every one of the above (and any future regression in escrow/wallet logic) can only be caught by manual QA or by users in production.
- **Zero server-managed cascade risk found** on money/audit-critical relations -- `Escrow` , `Transaction` , and `AdminAuditLog` rows cannot be silently deleted via a parent cascade; this is a genuinely good pattern, not a finding.

K. Recommended Implementation Roadmap

Phase 0 -- Critical fixes (security, correctness, money-integrity)

Feature	Reason	Dependencies	Frontend	Backend	Database	Risk	Order
Fix admin 2FA bypass	Highest-severity auth hole	none	none	<code>src/lib/auth</code> no one-condition fix	none	Low (small, well-understood change)	1
Fix hCaptcha (real verify + render widget)	Signup has zero bot protection	hCaptcha account already exists	render <code><Captcha></code> + wire state on register page	implement real <code>captcha.verify</code>	none	Low	2
Fix escrow milestone state-machine bypass	Violates the app's own core invariant	none	none	route through <code>insertTransition</code> milestone-release route	none	Medium (touches a money-adjacent completion path -- needs careful testing)	3

Feature	Reason	Dependencies	Frontend	Backend	Database	Risk	Order
Enforce admin RBAC permissions	Staff accounts have unrestricted access today	permissions already exists	staff-role UI unaffected	pass correct <code>permissions</code> <code>permissionsAdmin()</code> in Section I	none	Medium (broad but mechanical change; must not lock out the true owner account)	4
Fix wallet debit race (atomic decrement)	Theoretical double-spend under concurrency	none	none	switch <code>escrows/route.ts</code> <code>admin/withdrawals/[id]/route.ts</code> <code>{decrement}</code>	none	Low-Medium	5
Verify/fix CREDENTIALS	Possible silent crypto misconfiguration	none	none	confirm the deployed key is valid 64-hex; rotate if not (re-encrypt any affected data)	none	Medium (touches live encrypted data -- needs a careful, verified rollout)	6
Fix bank-wire withdrawal	A shipped payment feature is completely non-functional	mirrors existing crypto-withdrawal pattern	minor payload fix or none	branch <code>withdraw/route.ts</code> method or split endpoint	none	Low-Medium	7

Phase 1 -- Complete existing unfinished features

Feature	Reason	Dependencies	Frontend	Backend	Database	Risk	Order
Register the sweep task on the live server	Offer/escrow deadlines aren't proactively processed	<code>scripts/sweep</code> already exists	none	none (ops task)	none	Low	1
Fill in <code>AdminAuditLog</code> gaps	Compliance/forensic gap on sensitive actions	<code>auditLog()</code> already exists	none	add call to ~13 identified route files	none	Low	2
Add server-side size check to data-URI upload path	Bypassable upload cap	none	none	mirror the existing FormData-path check	none	Low	3
Fix forgot-password client error handling	Silent failure reported as success	none	add <code>res.ok</code> check	none	none	Low	4
Fix "Saved" listings count	Always-zero broken metric	decide: reuse <code>Watchlist</code> or add <code>SavedListing</code>	none if reusing Watchlist	update the query	possibly none	Low	5
Wire or remove phone-verification dead code	Misleading UX / dead code maintenance	needs an SMS provider decision if wiring for real	<code>settings/ver</code>	existing routes, or delete them	none	Low-Medium (depends on scope decision)	6

Feature	Reason	Dependencies	Frontend	Backend	Database	Risk	Order
Fix/trim public /docs and /sitemap pages	Broken link + fictional API docs	none	update page content	generate real sitemap link	none	Low	7

Phase 2 -- Critical missing business functionality

Feature	Reason	Dependencies	Frontend	Backend	Database	Risk	Order
Subscription cancellation/downgrading today; real support/billing exposure	No path exists	existing subscription/payment infra	settings/subscription UI	cancel/downgrade endpoint	none (uses existing SubscriptionPlan)	Medium	1
Automated test suite (core money paths first)	Zero regression protection on financial logic	test framework choice (Vitest recommended, matches existing tsx/TS tooling)	none	test files for fees.ts, escrow-state-machine.ts, credentials-crypto.ts, then integration tests for escrow/wallet routes	test DB	Low risk to add, high value	2
Real SMS phone verification (or formal removal)	Misleading current state	SMS provider account (e.g. Twilio) if implementing for real	wire existing UI step	wire existing routes to real SMS send	none	Medium	3

Phase 3 -- Operational improvements

Feature	Reason	Dependencies	Frontend	Backend	Database	Risk	Order
Error tracking / APM integration (Sentry or equivalent)	No production alerting today	account/SDK setup	wrap global-error.tsx	wrap server error paths, logger.ts	none	Low	1
Reconcile/clean up deployment configs	Confusing, contradictory, partly-broken configs	decide: keep Docker path or delete it	none	fix or remove docker-compose.yml, vercel.json; fix remaining stale-path scripts	none	Low	2
Add per-service health checks to status page	Currently misleading	none	update StatusClient	add real Socket.IO/escrow-processing checks to /api/health	none	Low	3
Auto-advance dispute phases on deadline expiry	Disputes can stall indefinitely	sweep task must be running (Phase 1)	none	extend internal/sweep-dispute-deadlines	possibly add deadline fields if not already present	Medium	4

Phase 4 -- Advanced features

Feature	Reason	Dependencies	Frontend	Backend	Database	Risk	Order
Wire <code>EscrowFlag</code> and at least one real feature	Currently fully built but gates nothing	none	conditional rendering using <code>isFeatureEnabled()</code>	none (helper exists)	none	Low	1
Build an NPS reporting view	Data is being collected into a black hole	<code>NpsResponse</code> already populated	new admin page	new read endpoint	none	Low	2
Consolidate duplicate admin components	Maintainability, not correctness	none	delete the 7 identified orphaned components after confirming no future plans for them	none	none	Low	3
Bidirectional trust score (penalties for lost disputes/cancellations)	Currently one-directional	dispute resolution already implemented	none	add penalty logic to the dispute-resolve (seller-loses) and cancellation paths	none	Medium (product/policy decision needed on exact penalty amounts)	4
Two-tier/granular roles beyond USER/ADMIN (e.g. MODERATOR)	Currently binary at the DB level; <code>StaffRole</code> is the only granularity and it's under-enforced	Phase 0's RBAC-enforcement fix should land first	staff UI already mostly supports this	schema addition if a true DB-level role tier is wanted (optional -- <code>StaffRole</code> may be sufficient once enforced)	possible enum extension	Medium	5

L. Final Master Checklist

Critical

- [] Remove the admin-role short-circuit that bypasses TOTP 2FA on password login (`src/lib/auth.ts:113`)
- [] Implement real hCaptcha `siteverify` call; render `<Captcha>` and wire `captchaToken` on the register page
- [] Route the milestone-release escrow completion through `assertTransition` instead of setting status directly
- [] Enforce `StaffRole` permissions on every sensitive admin route (especially `MANAGE_FINANCE`, `MANAGE_USERS`)
- [] Switch wallet debit paths (escrow purchase, withdrawal approval) to atomic `{decrement}`
- [] Verify the live `CREDENTIALS_ENCRYPTION_KEY` is a valid 64-hex-char key; rotate/fix if not
- [] Fix bank-wire withdrawal payload/schema mismatch so submissions actually succeed

High Priority

- [] Register a Windows Scheduled Task for `api/internal/sweep` (offer expiry, escrow flags, referral rewards, saved-search alerts)
- [] Build subscription cancellation/downgrade (currently completely missing)
- [] Stand up an automated test suite starting with `fees.ts`, `escrow-state-machine.ts`, `credentials-crypto.ts`, and core escrow/wallet API routes

- [] Decide and act on phone verification: wire it up with a real SMS provider, or remove the dead code/misleading UI copy
- [] Fill `AdminAuditLog` gaps: bank accounts, platform settings, feature flags, KYB decisions, escrow-manager-emails, transfer policies, canned responses, email templates, admin's own 2FA toggle
- [] Add rate limiting to admin mutation routes, especially TOTP verification during 2FA setup

Medium Priority

- [] Add server-side size validation to the data-URI upload path (`api/upload/listing`)
- [] Fix forgot-password client to surface real errors instead of always showing "sent"
- [] Fix or reconsider `PromoRedemption`'s delete-on-use pattern (potential replay)
- [] Make the duplicate-pending-offer check race-safe
- [] Convert the three identified config-level `Float` money fields to `Decimal`
- [] Add a `@@index` to the `Dispute` model
- [] Add a top-level try/catch to `api/internal/sweep`
- [] Integrate an error-tracking/APM service (Sentry or equivalent)
- [] Reconcile deployment configs: fix or remove `docker-compose.yml` and `vercel.json`; fix remaining stale `accsmarkets.org` -path scripts (`ecosystem.config.js`, `start-prod.ps1`, `scripts/sweep.bat`, `scripts/backup.bat`, `scripts/run-marketing-sweep.ps1`, `scripts/setup-backup-task.bat`, `scripts/setup-sweep-task.bat`)
- [] Auto-advance dispute phases on evidence/mediation deadline expiry
- [] Add real per-service checks to the public status page

Low Priority

- [] Fix the always-zero "Saved" listings dashboard metric
- [] Fix or remove the broken `/sitemap.xml` link on the public sitemap page
- [] Trim the public `/docs` page to only document real `/v1/*` endpoints (or build the missing ones)
- [] Wire the `FeatureFlag` system into at least one real product feature
- [] Build an admin-facing NPS response viewer
- [] Remove or consolidate the 7 identified orphaned/duplicate admin & dashboard components
- [] Add a penalty path to trust score for lost disputes/cancellations (currently one-directional)
- [] Add notifications for offer-cancel, withdrawal-request-creation, and deposit-creation events (currently silent)

This report is a snapshot as of 2026-09-16. It reflects direct code inspection (not README claims) across 6 parallel deep-dive workstreams covering the database schema, all API routes, all frontend pages/components, admin authorization, core business logic, and integrations/deployment. No code was modified as part of producing this report.